

Semantic Game Recommendation System with Production-Grade MLOps Infrastructure

Qurat Ulain Ahmed
Reg: 2024526

Maimoona Saboor
Reg: 2024270

Atta ur Rahman
Reg: 2024122

Labiba Ahmad
Reg: 2024260

Abstract—Recommender systems are a critical enabler of personalized content discovery in large-scale digital ecosystems, yet game recommendation remains underexplored relative to domains such as movies or music. The rapid expansion of gaming catalogs across platforms like Steam and RAWG introduces severe information overload, while existing approaches suffer from cold-start limitations, lack of semantic modeling, and poor interpretability. This work presents a hybrid, content-aware game recommender system that integrates multi-source metadata from Steam and RAWG through a unified preprocessing pipeline. Game similarity is modeled using a weighted combination of TF-IDF cosine similarity, Latent Semantic Analysis via Truncated SVD, and a Bayesian-smoothed popularity prior. Additional machine learning components include an XGBoost regression model for popularity prediction, a multi-label One-vs-Rest classifier for player archetype profiling, and KMeans clustering for latent genre discovery. The hybrid recommender achieves Tag Jaccard@10 of 0.467, outperforming both TF-IDF and SVD baselines, while the popularity regressor achieves $R^2=0.286$ and the archetype classifier reaches macro F1=0.846. The system is deployed as a FastAPI REST service with Docker containerization and a GitHub Actions CI/CD pipeline, confirming its readiness for real-world production environments.

Index Terms—Video Game Recommendation, Machine Learning, Content-Based Filtering, MLOps, FastAPI, TF-IDF, Latent Semantic Analysis, Clustering, Regression, Multi-Label Classification, Docker, CI/CD

I. INTRODUCTION

This project falls under the **Entertainment & Media** domain. Recommender systems have become a fundamental component of modern digital platforms, enabling personalized content discovery in domains such as e-commerce, media streaming, and digital gaming. In the context of video games, the rapid expansion of large-scale catalogs across platforms like Steam and RAWG has created a significant challenge for users to efficiently discover relevant titles aligned with their preferences. Unlike movies or books, game discovery is complicated by the highly heterogeneous nature of game metadata, which spans structured attributes, user-generated tags, textual descriptions, and platform-specific engagement signals. This diversity of signals makes naive filtering approaches insufficient and motivates the development of more sophisticated, multi-modal recommendation frameworks. This work presents a hybrid, content-aware game recommender system that integrates large-scale multi-source game data with machine learning techniques to provide accurate, explainable, and scalable game recommendations.

Code & deployment artifacts: GameRec-MLOps-Pipeline-ML-AI221

The importance of accurate game recommendation extends beyond user convenience. Platform operators rely on recommendation quality to drive user engagement, reduce churn, and surface high-quality but lesser-known titles. For independent developers in particular, an effective recommender can directly impact the commercial viability of their titles by connecting them with users whose preferences align with niche or novel game mechanics. Furthermore, as catalogs grow into the hundreds of thousands, keyword-based search becomes an increasingly poor substitute for semantic understanding of game content. The proposed system addresses this need by combining lexical and latent semantic representations, enabling the discovery of semantically related games even when they share no exact tag or genre overlap. Beyond recommendation, the system supports auxiliary tasks such as player profiling, pre-launch popularity estimation, and temporal analysis of release patterns, providing a broader analytical framework for understanding the gaming ecosystem.

A. Related Work

Recent work on game recommender systems has explored both content-based and collaborative filtering approaches using large-scale datasets from platforms like Steam. Calvin [1] demonstrates a Steam-based recommender that applies TF-IDF vectorization over game descriptions and tags, followed by cosine similarity to identify similar titles. This approach handles cold-start scenarios effectively but is limited by its reliance on surface-level lexical matching, failing to capture semantic relationships between games that describe similar gameplay using different terminology. Sumeet [2] extends this foundation by incorporating collaborative signals derived from user interaction data such as playtime and ratings, framing the recommendation task as a user-item similarity problem. While effective for active users with rich interaction histories, this approach suffers from well-known cold-start limitations and requires large volumes of user data to generalize. Madbonze [3] presents a hybrid video game recommender combining content-based features with popularity signals, demonstrating that blending multiple similarity sources improves recommendation diversity and reduces popularity bias. However, this system lacks explainability mechanisms and does not incorporate probabilistic smoothing over sparse rating distributions.

B. Gap Analysis

Collectively, prior works establish the viability of content-based and hybrid approaches for game recommendation but highlight persistent limitations: dependence on exact lexical matches, cold-start sensitivity, absence of semantic modeling, and lack of interpretability. No existing publicly available system simultaneously integrates multi-source data from both Steam and RAWG, applies Latent Semantic Analysis to bridge the gap between lexical and semantic similarity, and provides an explainability module alongside player profiling and popularity prediction. Furthermore, prior works do not address temporal seasonality in game releases, which is a practically useful signal for release planning and catalog management. Production deployment considerations such as containerization, CI/CD pipelines, and REST API design are similarly absent from the academic literature on game recommenders, leaving a significant gap between research prototypes and deployable systems.

C. Problem Statement

The following research questions are addressed in this study.

- 1) Does combining TF-IDF and SVD-based similarity outperform either method alone for game recommendation?
- 2) Can game popularity be predicted from content features alone, without access to post-release user interaction data?
- 3) Can player archetypes be accurately inferred from game metadata using multi-label classification?
- 4) Do latent game genres recoverable through unsupervised clustering offer finer resolution than official genre labels?
- 5) Do release timing patterns exhibit statistically exploitable seasonality across different game themes?

D. Contributions

This work makes the following contributions. A unified multi-source data integration pipeline is designed to merge Steam and RAWG datasets despite schema mismatches, normalizing over 79,000 games into a single knowledge base. A hybrid recommender is proposed that combines TF-IDF cosine similarity, LSA-based semantic embeddings, and a Bayesian-smoothed popularity prior, outperforming individual baselines on both Genre Overlap@10 and Tag Jaccard@10. Auxiliary ML components are developed for popularity regression, player archetype classification, and latent genre discovery via KMeans clustering. An explainability module is introduced that surfaces shared genres, tags, and influential TF-IDF features per recommendation. A production-grade deployment is implemented using FastAPI, Docker, and a GitHub Actions CI/CD pipeline.

II. DATA COLLECTION AND PREPROCESSING

A. Datasets

The proposed system draws data from two primary sources: the Steam catalog and the RAWG Video Games Database. The Steam dataset is provided in CSV format and includes

structured attributes such as game titles, release dates, descriptions, genres, and user engagement metrics. A formatting inconsistency in the Steam CSV (39 column headers, 40 values per row) causes the AppID column to be interpreted as the dataframe index, resulting in a systematic shift between column labels and corresponding values. This misalignment is corrected through selective column loading via `usecols`, followed by manual renaming and filtering by a minimum review count threshold to remove low-signal entries. The RAWG dataset is provided in JSON Lines (JSONL) format, totaling approximately 4 GB with nearly 890,000 game entries. Due to its scale, a streaming approach is adopted where records are processed sequentially, retaining only those with a minimum number of user ratings (votes count), not a rating value threshold. Nested attributes such as genres, tags, and platforms (represented as lists of dictionaries in RAWG) are flattened into standardized string lists for compatibility with downstream processing. After preprocessing, the Steam dataset retains 65,391 entries and the RAWG dataset 26,606.

B. Multi-Source Integration

Integrating Steam and RAWG introduces schema mismatches, naming inconsistencies, and varying data quality. A normalized key (`norm_key`) is constructed for each game by converting titles into lowercase alphanumeric strings, enabling matching despite differences in punctuation or spacing. An outer join strategy ensures games unique to either source are retained. Overlapping attributes are consolidated through set union for genres, tags, and platforms; RAWG descriptions are prioritized due to their richness, with fallback to Steam’s “About the game” field; and rating signals from both sources are merged into a unified Bayesian-smoothed popularity score:

$$\text{Score} = \frac{v}{v+m} R + \frac{m}{v+m} C \quad (1)$$

where R is the item’s average rating, v is the number of votes, m is the minimum vote threshold, and C is the global average rating across all items.

C. Exploratory Data Analysis

Genre distribution analysis reveals a strong dominance of Indie, Action, and Adventure categories, indicating a skew toward mainstream genres in the merged catalog. The popularity distribution follows a near-normal pattern centered around mid-to-high ratings, with most games receiving moderately favorable feedback. Figure 1 shows the top 10 genres and Figure 2 shows the Bayesian popularity distribution. Prior to joining, both datasets are deduplicated on the normalized key, retaining the highest-engagement record per title to ensure merge stability.

III. METHODOLOGY

A. Overall System Workflow

The system follows a modular pipeline illustrated in Figure 3. Raw data is ingested from Steam and RAWG, preprocessed into a unified catalog, and passed through a feature engineering stage that produces TF-IDF and LSA representations. These

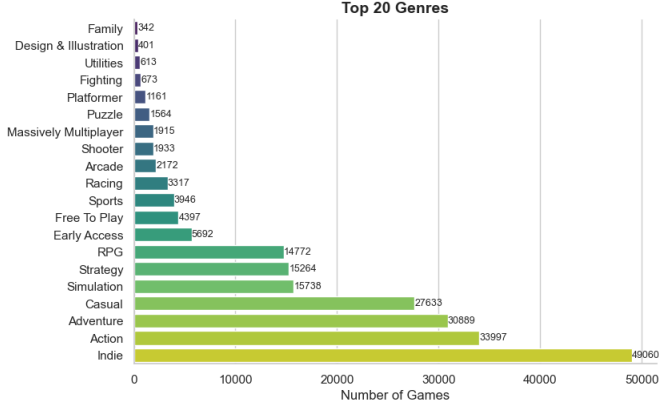


Fig. 1. Top 10 genres by frequency in the merged Steam and RAWG catalog. Indie, Action, and Adventure collectively represent more than half of all cataloged games.

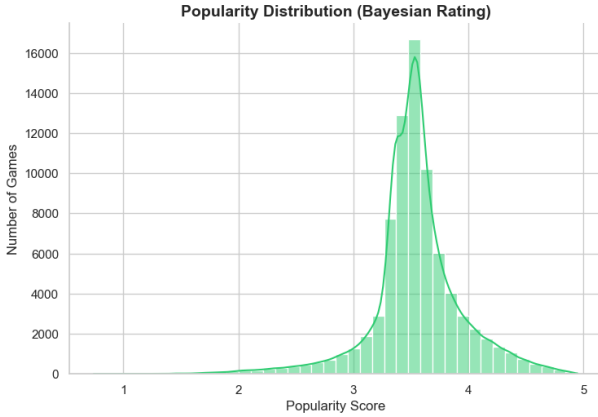


Fig. 2. Histogram of Bayesian-smoothed popularity scores across the merged catalog. The distribution is approximately normal, centered between 3.0 and 4.0 on the 0–5 scale, with a standard deviation of ≈ 0.39 .

representations feed into four model components: a hybrid content-based recommender, an XGBoost popularity regressor, a multi-label player archetype classifier, and a KMeans genre clustering module. Trained components are serialized to twelve distinct artifact files including the catalog, TF-IDF vectorizer, sparse matrix, SVD model, LSA matrix, popularity vector, archetype classifier, popularity regressor, KMeans model, cluster cards, labeling rules, and a seasonality JSON, allowing any downstream service to reconstruct the full system in under one second without reprocessing either raw dataset. Inference is exposed through a FastAPI REST API, and the entire system is containerized via Docker with automated testing and deployment managed through a GitHub Actions CI/CD pipeline.

B. Feature Engineering

Each game is represented as a single text blob combining multiple metadata fields with repetition-based weighting: genres are repeated 3 times, tags 4 times, categories 2 times, and descriptions are included once, ensuring that more informative

fields contribute more strongly to similarity calculations. Tags receive the highest weight because they are crowd-sourced and highly specific, constituting the strongest per-game signal in the catalog. All text is lowercased and stripped of special characters; multi-word labels are joined with underscores (e.g., *open_world*) to preserve them during tokenization. The processed text blobs are converted to numerical vectors using TF-IDF with: bigrams (`ngram_range=(1, 2)`), sublinear TF scaling (`sublinear_tf=True`), minimum document frequency `min_df=2`, and a vocabulary cap of 50,000 features. A content richness factor, defined as the number of tags per game capped at 8 and scaled to $[0, 1]$, penalizes games with sparse representations.

C. Latent Semantic Analysis via Truncated SVD

TF-IDF treats every term as independent, causing games that describe similar gameplay with different terminology (e.g., *sandbox* vs. *open_world*) to appear unrelated. Truncated Singular Value Decomposition (SVD) is applied to the TF-IDF matrix to compress high-dimensional sparse data into 128 latent dimensions:

$$T \approx U \Sigma V^T \quad (2)$$

All game vectors are subsequently L2-normalized so that cosine similarity reduces to a simple dot product, enabling efficient catalog-wide comparison through matrix operations.

D. Hybrid Similarity-Based Recommender

The hybrid recommender scores every game against a query using a weighted combination of three signals:

$$\text{score} = w_{\text{lsa}} \cos_{\text{SVD}} + w_{\text{txt}} \cos_{\text{TF-IDF}} + w_{\text{pop}} \text{popularity} \quad (3)$$

Weights are set to $w_{\text{lsa}} = 0.55$, $w_{\text{txt}} = 0.30$, and $w_{\text{pop}} = 0.15$, giving greatest importance to latent semantic similarity while preserving direct term overlap and quality signals. A second inference path, preference-driven recommendation, accepts a list of liked games and aggregates their LSA vectors into a virtual user profile, against which the full catalog is scored. The popularity weight is increased in this mode to favour broadly appreciated titles when the input signal is ambiguous. For query-based (preference-driven) recommendation, the popularity weight is increased to favour widely appreciated games when the input is ambiguous, and a content richness factor further penalizes games with very few tags

E. Explainability Module

An explainability module is integrated to identify the shared genres, tags, and influential TF-IDF features contributing to each recommendation score. For each recommended game pair, the module identifies overlapping genre labels, computes Jaccard similarity over tag sets, and extracts the top TF-IDF feature contributions to the similarity score, providing traceable justifications that directly address the black-box limitation of standard recommender systems.

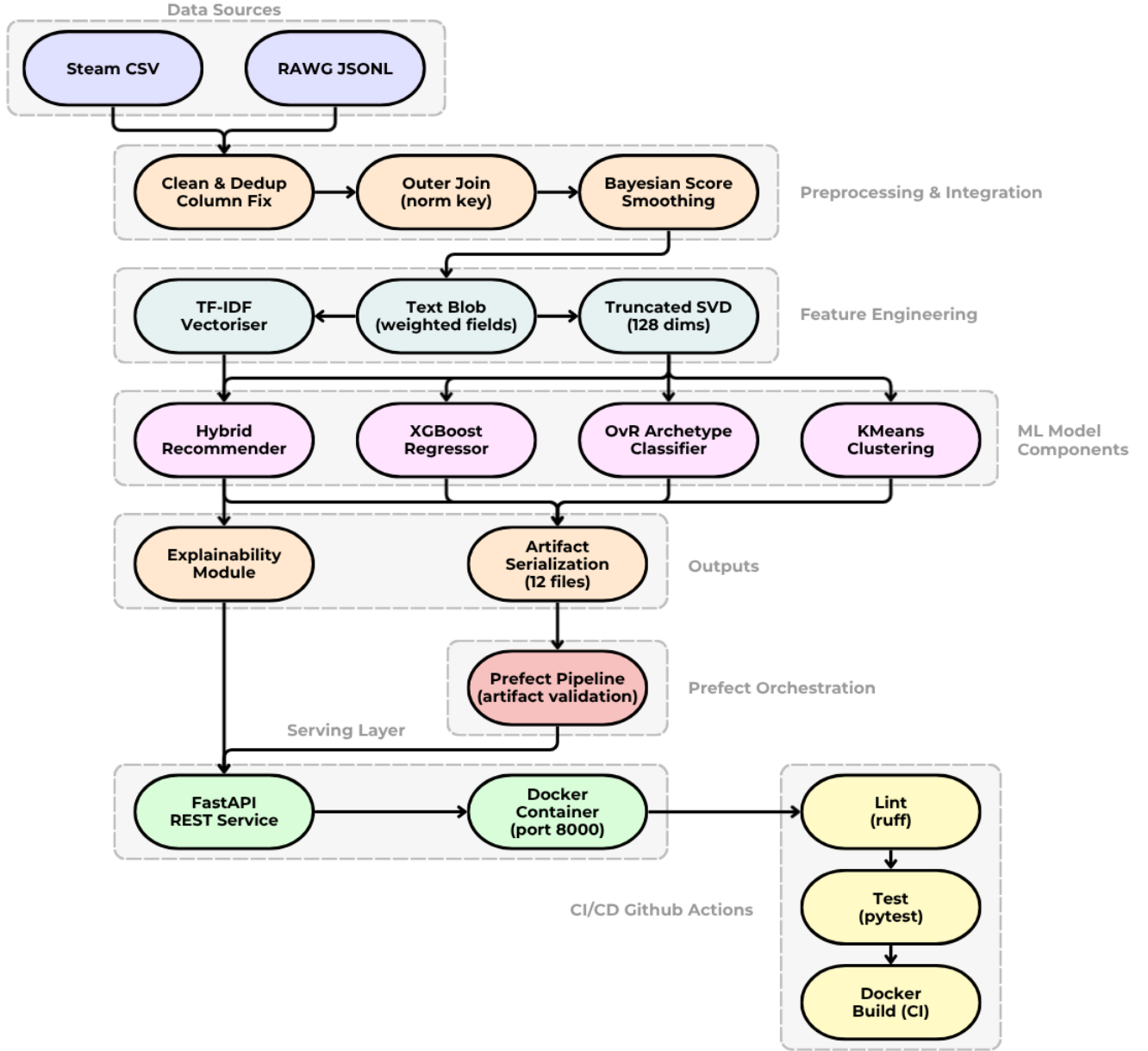


Fig. 3. Complete end-to-end system architecture. The offline training pipeline (notebook) produces 12 serialized artifacts, validated by a Prefect orchestration flow before the FastAPI service starts. The FastAPI application runs inside a Docker container with artifacts mounted as a read-only volume. A parallel GitHub Actions CI/CD pipeline gates code quality through lint, test, and smoke Docker build stages independently of the serving path.

F. Player Archetype Classification

A multi-label One-vs-Rest Logistic Regression classifier is trained on weakly supervised labels to predict eight player archetypes from game metadata: Explorer, Narrative Nerd, Tinkerer, Trophy Hunter, Thrill-Seeker, Grinder, Speedrunner, and Competitor. Weak supervision labels are generated using hand-crafted heuristics based on the presence of specific tags associated with each playstyle. The classifier is trained on 128-dimensional LSA embeddings, allowing it to generalize to games that do not contain the exact labeling keywords.

G. Popularity Regression

Several regression models are evaluated; XGBoost is selected as the final model based on held-out validation performance. An XGBoost regression model is trained to predict the Bayesian-smoothed popularity score from content features alone, simulating a pre-launch cold-start scenario. The feature set combines 128-dimensional LSA embeddings, a normalized release period feature, structural metadata features (tag count, description length), and binary attribute flags (multiplayer support, platform indicators). The model is configured with

learning rate 0.03, maximum depth 7, $\approx 85\%$ row and feature subsampling, and early stopping with patience 50 on a 10% held-out validation slice of the 80% training split.

H. Latent Genre Discovery via KMeans

KMeans clustering is applied to the L2-normalized 128-dimensional LSA matrix to discover latent game genres without label supervision. The optimal cluster count is selected by evaluating the silhouette score over $k \in \{8, 10, 12, 14, 16\}$ on a 5,000-point random subsample:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (4)$$

Each cluster is then interpreted through a lift-based tag analysis:

$$\text{Lift} = \frac{P(\text{tag} \mid \text{cluster})}{P(\text{tag})} \quad (5)$$

Tags with the highest lift values are selected to construct a semantic description for each latent genre.

I. Temporal and Seasonality Analysis

Release dates spanning 2014–2024 are used to compute monthly seasonal indices for five themes (Horror, Strategy, RPG, Sports, Racing):

$$\text{SeasonalIndex}_m = \frac{\text{Average releases in month } m}{\text{Average releases per month}} \quad (6)$$

A seasonal-naive forecaster predicting monthly fractions of annual releases is compared against a flat baseline on the 2024 holdout year, with performance measured by MAE in percentage points per month.

J. Experimental Settings

The system is implemented in Python using scikit-learn for TF-IDF, SVD, and classification components, XGBoost for regression, and FastAPI for the REST API. All random seeds are fixed for reproducibility. Key configuration settings are summarized in Table I.

IV. RESULTS

A. Recommender Comparison

The recommendation module is evaluated using two proxy metrics over 300 anchor games: Genre Overlap@10 (fraction of top-10 results sharing at least one genre with the query) and Tag Jaccard@10 (average Jaccard similarity of tag sets). Results for TF-IDF, SVD, and the proposed hybrid model are shown in Table II.

The hybrid model achieves the highest Tag Jaccard@10, demonstrating improved semantic alignment through its combination of lexical and latent representations. SVD slightly outperforms in Genre Overlap while both SVD and Hybrid significantly outperform TF-IDF in capturing meaningful semantic relationships.

TABLE I
CONFIGURATION SETTINGS FOR MAJOR SYSTEM COMPONENTS.

Parameter	Value
TF-IDF	
Max Features	50,000
N-gram Range	(1, 2)
Sublinear TF	Enabled
Min Doc. Freq.	2
SVD (LSA)	
Components	128
Random State	Fixed
XGBoost Regressor	
Learning Rate	0.03
Max Depth	7
Subsample	0.85
Early Stopping	50 rounds
KMeans Clustering	
Number of Clusters	16
Eval. Subsample	5,000

TABLE II
RECOMMENDER EVALUATION ON 300 ANCHOR GAMES. THE HYBRID MODEL ACHIEVES THE HIGHEST TAG JACCARD@10.

Method	Genre Overlap@10	Tag Jaccard@10
TF-IDF	0.956	0.448
SVD	0.978	0.456
Hybrid	0.977	0.467

TABLE III
POPULARITY REGRESSION RESULTS ON THE 20% TEST SPLIT. XGBOOST REDUCES MAE BY 13.9% AND RMSE BY 15.5% OVER THE MEAN BASELINE.

Model	MAE	RMSE	R ²	Time (s)
Mean Baseline	0.2618	0.3902	−0.000	0.00
XGBoost	0.2255	0.3297	0.286	81.61

B. Popularity Regression Results

The XGBoost regressor is evaluated against a mean-prediction baseline on the held-out test set. Results are shown in Table III.

XGBoost achieves a 13.9% reduction in MAE and 15.5% reduction in RMSE relative to the baseline. The baseline $R^2 \approx 0$ confirms it provides no predictive value, while XGBoost explains approximately 28.6% of variance in the popularity target—meaningful given the structurally narrow score distribution ($\sigma \approx 0.39$) that places a hard ceiling on achievable R^2 . Predicted vs. actual scatter, residual histogram, and feature

importance are shown in Figures 4–6.

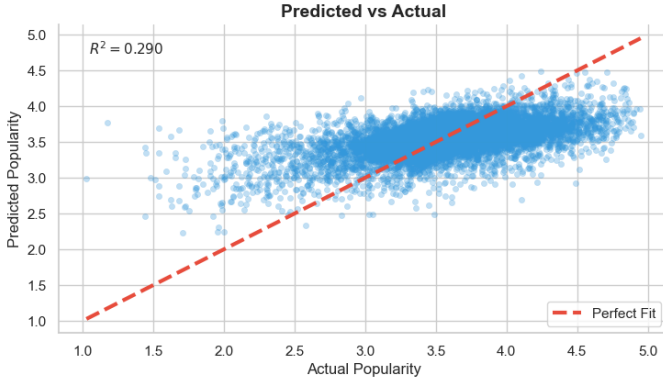


Fig. 4. Scatter plot of predicted vs. actual popularity for XGBoost on the 20% test set. Points near the diagonal indicate accurate predictions; clustering near the center reflects the narrow 3.0–4.0 target range.

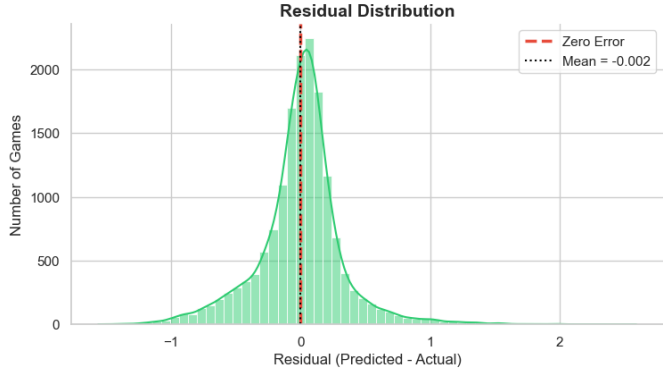


Fig. 5. Histogram of residuals (predicted minus actual) for the XGBoost popularity regressor. The distribution is centered near zero, indicating no strong directional bias.

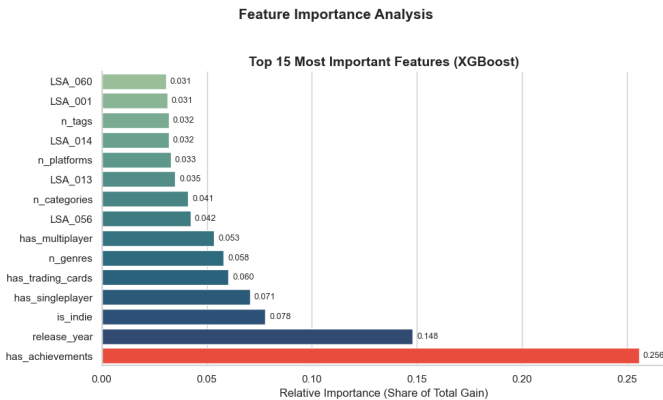


Fig. 6. XGBoost feature importances for popularity regression. LSA semantic features and the normalized release-period feature rank highest, confirming that both semantic content and release timing contribute meaningfully to predictions.

C. Player Archetype Classification

The multi-label classifier is evaluated on a 20% test split of 15,816 games. Per-class precision, recall, F1, and ROC-AUC are reported in Table IV; raw accuracy is not reported due to class imbalance.

The classifier achieves strong performance across majority archetypes; Explorer, Narrative Nerd, Tinkerer, and Thrill-Seeker all exceed $F1=0.93$, with ROC-AUC above 0.986. Trophy Hunter achieves the highest recall in the evaluation at 0.993 despite being a minority class, reflecting the strong and consistent tag signal associated with achievement-heavy games. Performance degrades for minority archetypes with sparser tag signals: Grinder, Speedrunner, and Competitor exhibit lower precision of 0.559, 0.611, and 0.523, due to lower label coverage and semantic overlap with adjacent archetypes. Despite lower precision, all three maintain recall above 0.90 and ROC-AUC above 0.97. Qualitative validation confirms correctness: for *The Witcher 3: Wild Hunt*, top predicted archetypes are Narrative Nerd ($p=1.000$), Explorer ($p=0.999$), Trophy Hunter ($p=0.979$), and Tinkerer ($p=0.941$).

D. Clustering Results

KMeans is applied to the L2-normalized LSA matrix of 79,080 games. Table V summarizes silhouette scores across candidate values of k .

$k=16$ yields the highest silhouette score, producing 16 latent genre clusters with sizes ranging from 928 to 9,102 games. The low absolute silhouette values are expected: game metadata occupies a high-dimensional continuous space where genre boundaries are inherently fuzzy. Representative discovered latent genres are summarized in Table VI, and the PCA-reduced 2D cluster visualization is shown in Figure 7.

E. Seasonality Results

Seasonal release patterns are analyzed across 63,909 games spanning 2014–2024. Peak release months per theme are summarized in Table VII.

The Horror theme exhibits the strongest seasonal signal with an October peak of $1.60\times$ the theme’s monthly average. Sports and Racing follow a late-summer pattern peaking in September, consistent with real-world sporting calendars. Strategy and RPG exhibit comparatively flat profiles. Forecasting results on the 2024 holdout are summarized in Table VIII.

Horror is the only theme where the seasonal forecaster meaningfully outperforms the flat baseline, reducing shape error by 39.4%. For Strategy and RPG, the flat baseline outperforms the seasonal model, confirming that imposing a seasonal structure on near-uniform release distributions adds noise rather than signal. Seasonal index and forecaster comparison charts are shown in Figures 8 and 9.

V. DEPLOYMENT AND ORCHESTRATION

A. Containerization Workflow

The FastAPI inference service is containerized using a single-stage Docker image based on `python:3.12-slim`, keeping

TABLE IV
PER-CLASS CLASSIFICATION RESULTS ACROSS EIGHT PLAYER ARCHETYPES ON THE 20% TEST SPLIT (15,816 GAMES). MACRO AND MICRO F1 ARE REPORTED AS AGGREGATE SUMMARIES.

Archetype	Precision	Recall	F1	ROC-AUC	Test Positives
Explorer	0.940	0.942	0.941	0.986	7,307
Narrative Nerd	0.959	0.972	0.966	0.998	3,674
Tinkerer	0.917	0.956	0.936	0.993	3,988
Trophy Hunter	0.799	0.993	0.885	0.996	1,755
Thrill-Seeker	0.948	0.960	0.954	0.995	4,680
Grinder	0.559	0.880	0.684	0.960	1,914
Speedrunner	0.611	0.940	0.740	0.987	1,368
Competitor	0.523	0.910	0.665	0.974	1,272
Macro avg	—	—	0.846	—	—
Micro avg	—	—	0.889	—	—

Cluster Visualization (2D PCA Projection)

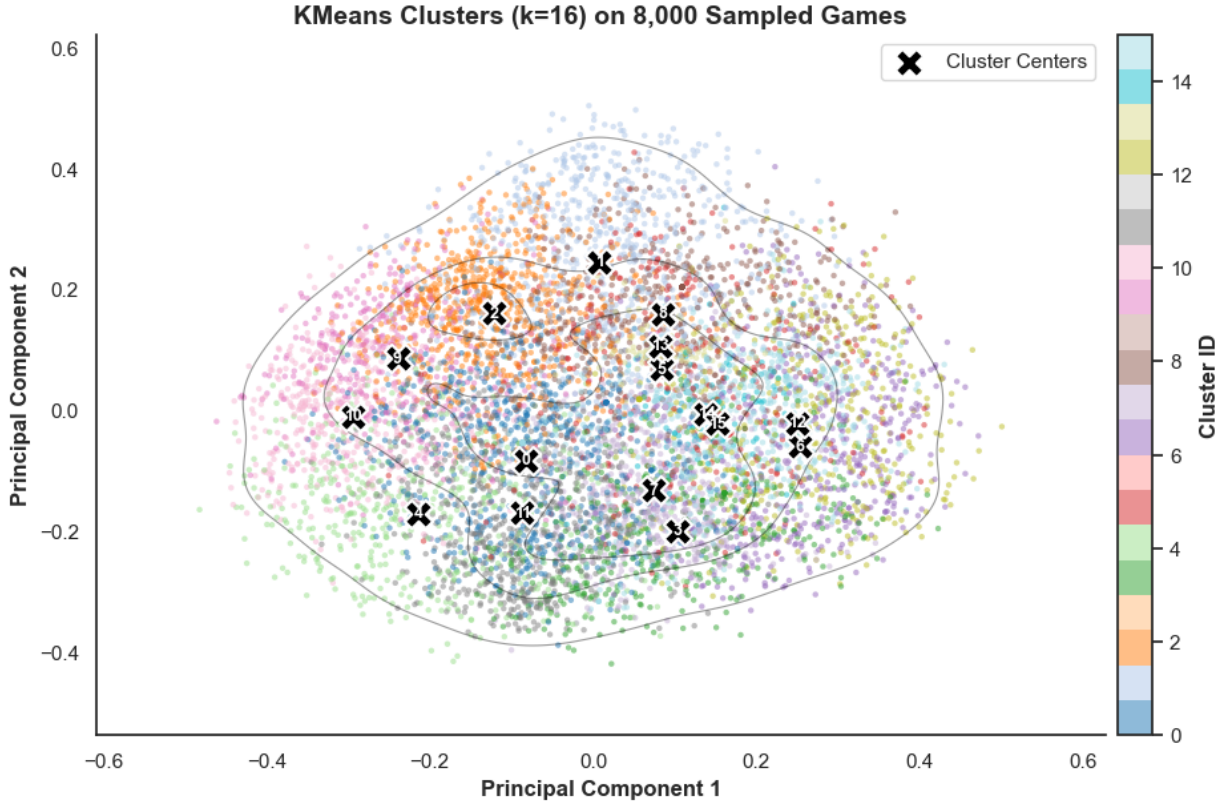


Fig. 7. PCA 2D scatter plot of 5,000 randomly sampled games colored by their KMeans cluster assignment ($k=16$). Each color represents one latent genre cluster. Distinct groupings are identifiable for well-separated clusters such as Precision Platformer and Boomer Shooter. Expected overlap appears for broader thematic categories that share surface-level genre tags—reflecting the fuzzy, multi-genre nature of modern games. The primary cluster interpretation is based on lift-driven tag analysis (Table VI) rather than visual separation alone.

the final image lightweight by excluding build-time dependencies. The application source and its Python requirements are copied into /app, and the container exposes port 8000, on

which Uvicorn serves the FastAPI application:

```
CMD ["uvicorn", "app.main:app",
      "--host", "0.0.0.0", "--port", "8000"]
```

TABLE V
SILHOUETTE SCORE VS. NUMBER OF CLUSTERS FOR KMEANS. $k=16$ IS SELECTED BASED ON THE HIGHEST SILHOUETTE SCORE.

k	Silhouette Score	Fit Time (s)
8	0.0542	18.8
10	0.0594	11.6
12	0.0651	18.6
14	0.0647	15.6
16	0.0703	16.5

TABLE VI
REPRESENTATIVE LATENT GENRES DISCOVERED BY KMEANS CLUSTERING. HIGH LIFT VALUES CONFIRM SEMANTICALLY COHERENT GROUPINGS WELL ABOVE CATALOG-WIDE BASELINE FREQUENCIES.

Cluster Name	Top Distinctive Tags (Lift)	Size
Sokoban / Solitaire	Sokoban (6.84 $\times$), Mahjong (6.63 $\times$), Solitaire (6.15 $\times$)	9,102
Boomer Shooter	Top-Down (8.97 $\times$), Boomer Shooter (8.11 $\times$), Bullet Hell (7.35 $\times$)	7,348
Precision Platformer	Precision Plat. (10.69 $\times$), Metroidvania (8.61 $\times$), 2D Plat. (8.64 $\times$)	6,009

TABLE VII
PEAK RELEASE MONTH AND SEASONAL INDEX PER THEME. HORROR EXHIBITS THE STRONGEST SEASONAL SIGNAL; STRATEGY AND RPG SHOW NEAR-FLAT PROFILES.

Theme	Peak Month	Seas. Index	Total Games
Horror	October	1.60 $\times$	9,420
Sports	September	1.28 $\times$	3,112
Racing	September	1.15 $\times$	2,430
Strategy	August	1.14 $\times$	12,572
RPG	October	1.14 $\times$	12,066

TABLE VIII
SEASONAL VS. FLAT BASELINE FORECASTER PERFORMANCE (MAE IN PERCENTAGE POINTS PER MONTH) ON 2024 HOLDOUT DATA. HORROR IS THE ONLY THEME WHERE THE SEASONAL MODEL MEANINGFULLY OUTPERFORMS THE FLAT BASELINE.

Theme	Flat (pp)	Seasonal (pp)	Improvement
Horror	1.20	0.73	+39.4%
Sports	1.49	1.49	+0.3%
Racing	1.33	1.54	-15.8%
Strategy	0.79	0.82	-3.0%
RPG	0.76	0.92	-21.8%

Trained ML artifacts are deliberately *not* baked into the image. Instead, `docker-compose.yml` mounts the host

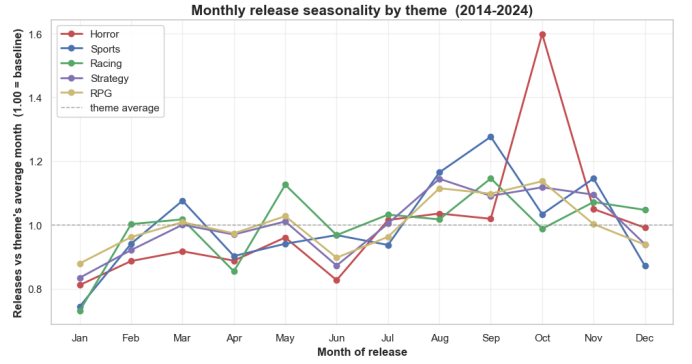


Fig. 8. Seasonal index line chart for five game themes over calendar months. Values above 1.0 indicate above-average release activity. Horror shows an isolated October peak at 1.60 $\times$ the theme’s average; Strategy and RPG remain near 1.0 throughout the year.

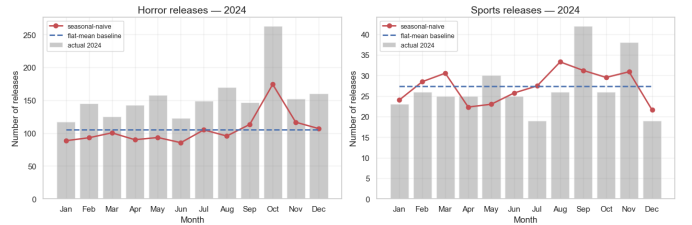


Fig. 9. Grouped bar chart comparing seasonal-naive vs. flat baseline forecaster MAE per theme on the 2024 holdout year. Lower bars indicate better performance. The seasonal model outperforms only for Horror; Strategy and RPG are better served by the flat baseline.

path `../artifacts/recommender` into the container at `/app/artifacts/recommender` as a read-only bind mount. This design separates the versioned model bundle from the application image, allowing artifacts to be updated independently without rebuilding or republishing the container. Compose also sets the build context to the repository root so the Dockerfile at `docker/Dockerfile` can access all project source files during the build step.

B. CI/CD Pipeline

Continuous integration is managed through a GitHub Actions workflow defined in `.github/workflows/ci.yml`, triggered on every push and pull request targeting the `main`, `master`, or `develop` branches. The pipeline executes three sequential jobs, each dependent on the success of its predecessor:

Lint installs development dependencies from `requirements-dev.txt` and runs `ruff check .` followed by `ruff format --check .` to enforce code style and catch static errors before any tests execute.

Test installs both development and production requirements, then invokes `pytest tests/ -q --tb=short` with the environment variable `GAMEREC_ARTIFACTS_DIR` pointing to a lightweight fixture directory, allowing inference-path tests to run without the full 4 GB artifact bundle.

Docker Build performs a smoke build of the production image via `docker build -f docker/Dockerfile -t`

`gamerec-api:ci` ., confirming that the image assembles correctly after every passing test run. No image is pushed to a registry and no cloud deployment is triggered; the build step serves as a build-time correctness gate rather than a continuous delivery stage. A secondary workflow, `discord-push.yml`, fires on the same branch events and posts a commit notification to a Discord webhook for team visibility. The complete three-stage pipeline is summarized in Figure 10.

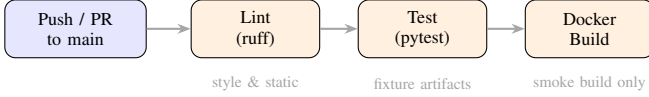


Fig. 10. GitHub Actions CI pipeline. The three jobs execute sequentially; each gate must pass before the next stage begins. No registry push or cloud deployment is performed.

C. Prefect Orchestration Flow

A Prefect pipeline, `gamerec_artifact_pipeline`, provides pre-serving validation of the offline-trained artifact bundle without triggering model retraining. Full retraining remains an intentionally offline process executed via the Jupyter notebook; the Prefect flow acts as a health-check gate confirming that the artifacts produced by the notebook are present and structurally valid before the FastAPI service is started. The flow is invoked manually via `python -m pipelines.flow` or the `make pipeline` Makefile target; no Prefect deployment schedule is configured.

The flow executes four tasks in order: (1) **validate_sources_and_environment** confirms that the Steam CSV and RAWG JSONL raw data paths are accessible and echoes environment configuration; (2) **verify_ml_artifacts** checks for the presence of all required and optional serialized files under `artifacts/recommender/`, parses `player_types_rules.json` to confirm structural validity, and sets a `ready_for_fastapi` flag; (3) **print_summary** logs the validation results as a structured summary block; (4) **send_discord_notification** dispatches an optional success or failure notification to a Discord webhook if a URL is configured. The orchestration flow is depicted in Figure 11.

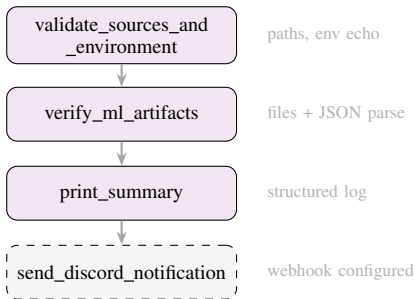


Fig. 11. `gamerec_artifact_pipeline` Prefect flow. The pipeline validates raw data sources and serialized ML artifacts without retraining; it serves as a pre-serving health-check gate invoked manually before launching the FastAPI container.

VI. DISCUSSION

The experimental results across all five system components confirm that integrating lexical, semantic, and quality signals produces better recommendations than any single method alone. The hybrid recommender’s Tag Jaccard@10 improvement over TF-IDF alone demonstrates that the SVD component successfully bridges the gap between lexically different but semantically related games. The near-equal Genre Overlap@10 across SVD and Hybrid indicates that both methods effectively capture genre-level structure, but the Hybrid model’s edge in Tag Jaccard reveals superior fine-grained semantic alignment. The explainability module further supports this finding by providing traceable justifications grounded in shared tags and genres, directly addressing the black-box limitation of standard recommender systems.

The popularity regression results illustrate both the promise and the fundamental limits of content-based predictive modeling. An R^2 of 0.286 is meaningful given that the target’s standard deviation of 0.39 compresses the effective prediction range to approximately one point on the 0–5 scale, placing a hard ceiling on achievable R^2 regardless of model sophistication. The model correctly identifies broad category-level predictors of popularity such as semantic richness, genre specificity, and release timing context, but cannot assess execution quality, which is the primary driver of outlier popularity in both directions. For pre-launch planning purposes, the MAE of 0.2255 is nonetheless operationally useful, providing a rough quality signal before any user feedback is available.

The player archetype classification results reveal an important asymmetry between majority and minority classes. Strong performance on Explorer, Narrative Nerd, and Thrill-Seeker reflects the clarity of the tag signals associated with these playstyles. Weaker precision on Grinder, Speedrunner, and Competitor reflects inherent semantic overlap with adjacent archetypes and smaller trigger tag sets. The clustering results demonstrate that official Steam genre labels are too coarse to capture meaningful distinctions: two games both labeled *Indie*, *Adventure* can occupy entirely different latent clusters reflecting fundamentally different playstyle identities. The temporal analysis reveals a clear bifurcation: Horror exhibits stable, exploitable seasonality while Strategy and RPG do not, and reporting these negative results is deliberate—a framework that only claims wins on favorable themes would be misleading for practical release planning.

A. Future Directions

The most impactful extension would be incorporating collaborative filtering signals: user-level play history, review patterns, and purchase behavior from public Steam APIs would enable truly personalized recommendations that adapt to individual histories. Incremental model updates should be investigated to eliminate full retraining as new games are released. Richer game metadata such as video trailers, screenshots, and developer reputation could improve popularity regression accuracy. For temporal analysis, modeling player demand rather than release supply would provide more commercially actionable insights.

Finally, a formal user study measuring recommendation satisfaction, diversity, and serendipity would provide ground-truth evaluation beyond the proxy metrics used here.

VII. CONCLUSION

This work presents a comprehensive end-to-end game recommender system integrating multi-source data ingestion, semantic feature engineering, supervised and unsupervised machine learning, and production-grade deployment infrastructure. The hybrid recommender combining TF-IDF cosine similarity, LSA-based latent similarity, and a Bayesian-smoothed popularity prior outperforms both single-method baselines across Tag Jaccard@10 and Genre Overlap@10, validating the design decision to blend lexical, semantic, and quality signals. The XGBoost popularity regressor demonstrates that game content carries genuine predictive signal independent of user feedback, achieving $R^2 = 0.286$ and $MAE = 0.225$ against a mean baseline with $R^2 \approx 0$. The multi-label player archetype classifier achieves macro F1 = 0.846 across eight archetypes, with majority classes exceeding F1 = 0.93, confirming the effectiveness of weak supervision on LSA embeddings. KMeans clustering successfully recovers 16 semantically coherent latent genres from 79,080 games, demonstrating that official genre labels are too coarse to capture meaningful distinctions between conceptually distinct game identities. Temporal analysis confirms that genre-level seasonality is real and measurable, with Horror reducing shape prediction error by 39.4%, while Strategy and RPG exhibit uniform release distributions that provide no exploitable seasonal signal. The system is deployed as a FastAPI REST service containerized with Docker and managed through a GitHub Actions CI/CD pipeline, demonstrating that the full research-to-production gap can be bridged within a single project scope.

REFERENCES

- [1] Calvin, “Steam Recommender System,” *Kaggle*, 2023. [Online]. Available: <https://www.kaggle.com/code/calven22/steam-recommender-system>. [Accessed: Apr. 30, 2026].
- [2] Sumeet, “Game Recommender System,” *Kaggle*, 2023. [Online]. Available: <https://www.kaggle.com/code/sumeet07/game-recommender-system>. [Accessed: Apr. 30, 2026].
- [3] Madbonze, “Video Game Recommender System,” *Kaggle*, 2023. [Online]. Available: <https://www.kaggle.com/code/madbonze/video-game-recommender-system/notebook>. [Accessed: Apr. 30, 2026].